Minimum Spanning Tree with Neighborhoods Problem

Arthur Péraud

Supervised by Prof. Fatiha Bendali and Dr. Jean Mailfert

November 2025

Contents

1 Introduction **3**

2 Problem definition **3**
2.1 From MST to MSTN . . . 3
2.2 An important property: MSTN cost versus MST cost . . . 4
2.3 Biconvex structure . . . 5

3 Heuristic algorithm for MSTN **5**
3.1 Overview and motivation . . . 5
3.2 Algorithmic description . . . 6
3.3 Convergence and optimality . . . 8

4 Python implementation and experiments **8**
4.1 Development progression: from discrete to continuous . . . 8
4.2 Code architecture and instance generation . . . 8
4.3 Visualization support . . . 8

5 Experimental results **9**
5.1 Detailed example: Behavior on a single instance . . . 9
5.2 Scaling and effectiveness: statistical evaluation over many instances . . . 12

6 MINLP formulations overview **14**
6.1 Mathematical formulation . . . 14
6.2 SEC-based formulation . . . 15
6.3 MTZ-based formulation . . . 16
6.4 Decomposition and mathheuristic . . . 16
6.5 Relationship to the implemented heuristic . . . 16

7 Future work: learning and advanced heuristics **17**
7.1 More advanced mathheuristics . . . 17
7.2 Machine learning for heuristic design . . . 17

8 Conclusion **18**

Bibliography **19**

Acknowledgments **19**

1 Introduction

Combinatorial optimization problems on graphs appear in many applications in operations research, network design, and logistics. A classical example is the Minimum Spanning Tree (MST) problem, where one is given a weighted connected graph and seeks a tree of minimum total edge length that spans all vertices. The standard MST model assumes that vertices are points embedded at known locations, and that edge lengths are fixed and known in advance, for example as Euclidean distances between vertex coordinates.

In many practical settings this assumption is not realistic. The exact position of a terminal may be only partially known, or constrained to lie within a region due to geometric tolerances, uncertainty, or flexibility in the final placement. In such cases each vertex is better modeled as a *neighborhood*, i.e., a convex region in the plane or in space, and a solution must choose one representative point from each neighborhood and connect those representatives by a spanning tree of minimum length. This leads to the *Minimum Spanning Tree with Neighborhoods* (MSTN) problem introduced and studied in detail by Blanco, Fernández, and Puerto [1].

The MSTN problem is significantly harder than the classical MST. Even in the planar Euclidean case the problem is NP-hard, and exact mixed-integer nonlinear programming (MINLP) formulations are required to solve only medium-size instances [1]. At the same time, the structure of the problem is rich: if the spanning tree is fixed, the best representatives can be found by solving a convex optimization problem; if the representatives are fixed, the best tree is a standard MST. This biconvex structure motivates heuristic and "mathheuristic" approaches that alternate between these two spaces.

The goal of this report is twofold. First, to present a concise overview of the MSTN problem and of the main mathematical programming formulations proposed in [1]. Second, to describe and experimentally illustrate a simple heuristic implementation developed in Python, based on alternating between MST computation and local relocation of representatives inside their neighborhoods. Numerical experiments on randomly generated instances show that this heuristic can substantially reduce the total tree length compared to the tree over neighborhood centers, with very modest computational effort.

2 Problem definition

2.1 From MST to MSTN

Let $G = (V, E)$ be a connected undirected graph with vertex set $V = \{1, \dots, n\}$ and edge set $E \subseteq \{\{i, j\} : i, j \in V,\ i < j\}$. In the classical Euclidean MST problem, each vertex $v \in V$ is associated with a fixed point $p_v \in \mathbb{R}^d$, and the length of edge $e = \{i, j\}$ is given by the distance

$$c_e = \|p_i - p_j\|.$$

A feasible solution is any spanning tree $T \subseteq E$, and the objective is to minimize

$$\sum_{e \in T} c_e.$$

In the MSTN problem, instead of fixed points, each vertex $v \in V$ is associated with a nonempty convex neighborhood $N_v \subset \mathbb{R}^d$. A solution must choose a representative point $y_v \in N_v$ for each v, and a spanning tree T on the chosen points. If $e = \{i, j\} \in E$, its length becomes the norm-based distance between y_i and y_j,

$$d_e(y) = \|y_i - y_j\|.$$

The total cost of a solution (Y, T), where $Y = \{y_v : v \in V\}$, is

$$d(T; Y) = \sum_{e \in T} d_e(y) = \sum_{\{i,j\} \in T} \|y_i - y_j\|.$$

The MSTN problem can therefore be stated as:

$$\text{minimize } d(T; Y)$$

subject to

- $y_v \in N_v$ for all $v \in V$,
- T is a spanning tree on the complete graph induced by the representatives Y.

Two extreme cases are worth noting. If each neighborhood degenerates to a singleton $N_v = \{p_v\}$, then MSTN reduces to the standard MST with fixed vertex positions. On the other hand, if neighborhoods are so large that they all intersect in a common nonempty region, an optimal solution is to pick the same representative point for all vertices, leading to a degenerate tree of cost zero.

2.2 An important property: MSTN cost versus MST cost

A fundamental observation for the MSTN problem is that the optimal cost of an MSTN is always *less than or equal to* the cost of the classical MST computed on the neighborhood centers. This follows directly from the fact that the neighborhood centers themselves constitute a feasible MSTN solution: by choosing y_v equal to the center of each N_v, we obtain a valid set of representatives, and computing the MST on these centers gives an upper bound on the optimal MSTN cost. Since the MSTN optimizer is free to choose *any* point within each neighborhood (including the center), it can achieve at least as good—and typically better—performance [1].

Formally, if c_v denotes the center of neighborhood N_v, and T^{MST} is the minimum spanning tree on the points $\{c_v : v \in V\}$, then

$$\text{MSTN}^* \leq \sum_{e \in T^{\text{MST}}} \|c_i - c_j\|,$$

where MSTN^* is the optimal MSTN cost. In practice, when neighborhoods are sufficiently large or overlapping, the MSTN cost can be *significantly* smaller than the MST cost over centers, since representatives can be moved closer together or even merged when neighborhoods intersect [1].

This property motivates the design of heuristics that start from the MST over centers and iteratively improve the solution by relocating representatives within their neighborhoods.

2.3 Biconvex structure

A key structural observation is that MSTN is *biconvex*. If the spanning tree x is fixed, the problem of optimizing over the representatives y and distances u is a convex program (in fact, a second-order cone program). Conversely, if the representatives y are fixed, the problem of selecting x becomes a classical MST, which can be solved by greedy algorithms such as Kruskal's algorithm. However, the joint problem in (x, y) is neither convex nor concave [1].

This biconvex structure is the foundation for both the exact decomposition methods and the heuristic approaches described in this report.

3 Heuristic algorithm for MSTN

3.1 Overview and motivation

The biconvex structure of the MSTN problem naturally suggests an alternating optimization approach. Given that:

- with positions fixed, computing the MST is a standard combinatorial problem solvable in polynomial time,
- with the tree fixed, updating the positions is a continuous adjustment step constrained by neighborhood geometry,

we can design a heuristic that iteratively alternates between these two subproblems.

The heuristic implemented in this project is based on the following intuitive idea. Start from the centers of all neighborhoods as initial representatives. At any iteration:

1. Compute an MST on the current representative points using Euclidean edge weights.
2. For each vertex, move its representative towards the barycenter (mean) of its neighbors in the current tree, but project the moved point back onto its neighborhood to preserve feasibility.
3. Repeat until the positions stabilize, or a maximum number of iterations is reached.

The hope is that by repeatedly shrinking edges along the tree, the total length of the MSTN decreases and converges to a configuration that is locally optimal with respect to this simple update rule.

Projection onto a square neighborhood. Consider a vertex i whose neighborhood N_i is a closed axis-aligned square of side s_i and center $C_i = (c_x, c_y)$. The feasible set is therefore

$$x \in [x_{\min}, x_{\max}] = \left[c_x - \tfrac{s_i}{2},\ c_x + \tfrac{s_i}{2}\right], \qquad y \in [y_{\min}, y_{\max}] = \left[c_y - \tfrac{s_i}{2},\ c_y + \tfrac{s_i}{2}\right].$$

Let $B^{(k)} = (b_x, b_y)$ denote the barycenter of the neighbors of vertex i at iteration k. The goal is to choose the representative $y_i^{(k+1)}$ in N_i that is closest (in Euclidean norm) to $B^{(k)}$:

$$\hat{y}_i^{(k+1)} = \mathrm{Proj}_{N_i}\big(B^{(k)}\big) = \arg\min_{P \in N_i} \|B^{(k)} - P\|_2.$$

Since the square is axis-aligned, this constrained minimization separates along the x and y coordinates. Writing $P = (p_x, p_y)$, we solve independently:

$$\min_{p_x \in [x_{\min}, x_{\max}]} (b_x - p_x)^2, \qquad \min_{p_y \in [y_{\min}, y_{\max}]} (b_y - p_y)^2.$$

The solutions are obtained by clamping the coordinates of $B^{(k)}$ to the interval bounds:

$$p_x = \begin{cases} x_{\min}, & b_x < x_{\min}, \\ b_x, & x_{\min} \le b_x \le x_{\max}, \\ x_{\max}, & b_x > x_{\max}, \end{cases} \qquad p_y = \begin{cases} y_{\min}, & b_y < y_{\min}, \\ b_y, & y_{\min} \le b_y \le y_{\max}, \\ y_{\max}, & b_y > y_{\max}. \end{cases}$$

Thus, the projection $\hat{y}_i^{(k+1)} = (p_x, p_y)$ is obtained simply by *clamping* each coordinate to the bounds of the square:

$$\mathrm{Proj}_{N_i}(b_x, b_y) = \big(\min(\max(b_x, x_{\min}), x_{\max}),\ \min(\max(b_y, y_{\min}), y_{\max})\big).$$

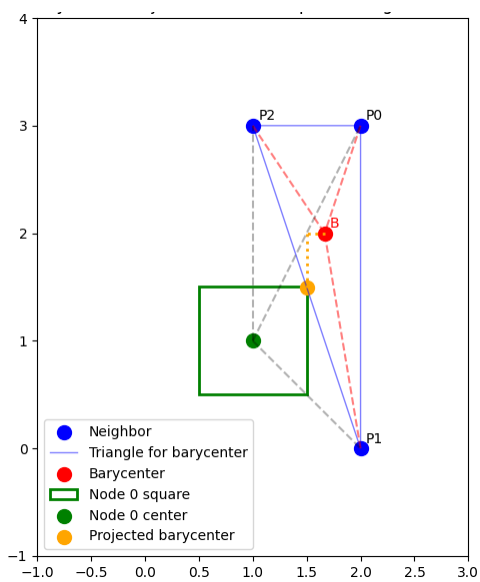

Figure 1: Geometric projection of a barycenter B onto the square neighborhood N_i.

3.2 Algorithmic description

The core heuristic function can be described as follows.

Initialization.

- Set the initial representative of each vertex i to the center of its neighborhood:

$$y_i^{(0)} := \text{center of } N_i.$$

- Compute the MST and its cost on these initial points using Kruskal's algorithm, obtaining a baseline cost C^{orig}.

Main loop. For iteration $k = 0, 1, 2, \ldots$ up to a maximum number of iterations:

1. Compute the current MST on the positions $\{y_i^{(k)}\}$ using Kruskal's algorithm. This yields a tree $T^{(k)}$ and its cost $C^{(k)}$.

2. For each vertex i, collect the indices of its neighbors in the tree $T^{(k)}$. The neighbor list is symmetric: if $(i, j) \in T^{(k)}$, then j is in the neighbor list of i, and i is in the neighbor list of j.

3. For each vertex i that has at least one neighbor:

   - Compute the arithmetic mean (barycenter) of its neighbors' positions:

     $$c_i^{(k)} = \frac{1}{|N_i^{(k)}|} \sum_{j \in N_i^{(k)}} y_j^{(k)},$$

     where $N_i^{(k)}$ is the set of neighbors of vertex i in tree $T^{(k)}$.

   - Project this barycenter onto the neighborhood N_i:

     $$\hat{y}_i^{(k+1)} = \mathrm{Proj}_{N_i}\big(c_i^{(k)}\big),$$

     where Proj_{N_i} is the Euclidean projection onto the convex set N_i.

   - Measure the displacement $\|\hat{y}_i^{(k+1)} - y_i^{(k)}\|$, and track the maximum displacement over all vertices.

4. Set the new positions:

   $$y_i^{(k+1)} := \hat{y}_i^{(k+1)}.$$

5. If the maximum displacement is below a tolerance `tol`, stop the iterations.

At termination, the function recomputes the MST on the final positions, and returns:

- the final positions $\{y_i^{(k_{\text{final}})}\}$,
- the list of MST edges,
- the final MST cost C^{final},
- the original cost at neighborhood centers C^{orig},
- the number of iterations performed.

This algorithm is a simple alternating heuristic in the sense that it alternates between a discrete optimization step (MST) and a continuous local improvement step (projected averaging). It is much simpler than the mathheuristic in [1], which solves exact convex subproblems in the continuous variables at each iteration, but it is also cheap to implement and very fast in practice.

3.3 Convergence and optimality

The heuristic does not guarantee global optimality, but it does guarantee convergence to a *stationary point* of the alternating optimization. At convergence:

- The tree T is optimal for the current positions (it is an MST).
- Each representative y_i is the projection of the barycenter of its neighbors onto N_i, which is a form of local optimality in the continuous subspace.

Because the objective function decreases (or remains constant) at each iteration and is bounded below by zero, convergence in objective value is guaranteed. However, the final solution may be a local minimum rather than a global one, especially if the initial tree structure is far from optimal. To mitigate this, one could use multiple random restarts or alternative initialization strategies, as proposed in [1].

4 Python implementation and experiments

4.1 Development progression: from discrete to continuous

In the early stage of the project, to simplify debugging and ensure correctness, the implementation was restricted to the case where all neighborhood centers and square sizes were integers, and neighborhoods were generated without overlapping. This made it easier to verify projections and MST calculations directly and provided a useful starting point for observing the algorithm's behavior.

Subsequently, the implementation was extended to support real-valued (floating-point) neighborhood centers, arbitrary side lengths, and (optionally) overlapping neighborhoods. This generalization is important for realism and enables experiments where the potential improvement of the heuristic is more significant, especially when representatives are allowed to coincide in intersection regions.

4.2 Code architecture and instance generation

The main data structure is the `GeoGraph` class, which stores all neighborhoods (square or circle), their parameters, the graph edges, and plotting utilities. Each neighborhood type provides a `Project(Point)` method that returns the projection of a given point onto the neighborhood (using coordinate-wise bounding for squares, or radial projection for circles).

Random instance generation is handled by helper functions such as `Random_point`, `Random_square`, `Random_square_non_overlapping`, and `Create_random_square_graph`, which allow the user to specify the number of neighborhoods, edge density, size and overlap constraints. Integer and real-valued parameters are both supported, depending on the experimental needs.

4.3 Visualization support

The implementation includes several Python functions for generating visualizations of neighborhoods, representative points, graph edges, and the progression of the heuristic algorithm. Details and examples of resulting plots are presented in Section 5.

5 Experimental results

5.1 Detailed example: Behavior on a single instance

For illustrative purposes, we first consider a random instance with 35 square neighborhoods on a complete graph (see Figure 2).

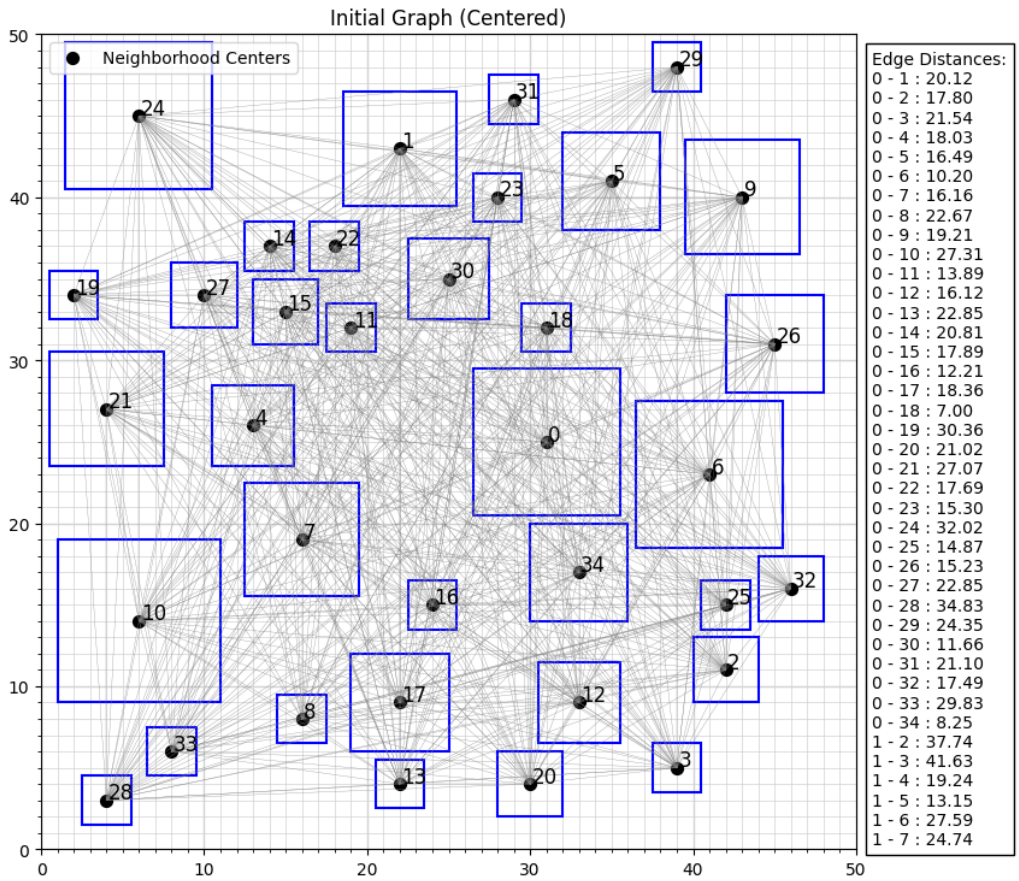

Figure 2: Initial graph with 35 square neighborhoods (blue boxes) and their centers (black dots). Gray lines show all edges of the complete graph.

Instance parameters:

- 35 neighborhoods
- Complete graph (all pairs connected)
- Squared neighborhoods of side in $[\![3; 10]\!]$
- Integer centers and integer side lengths, non-overlapping

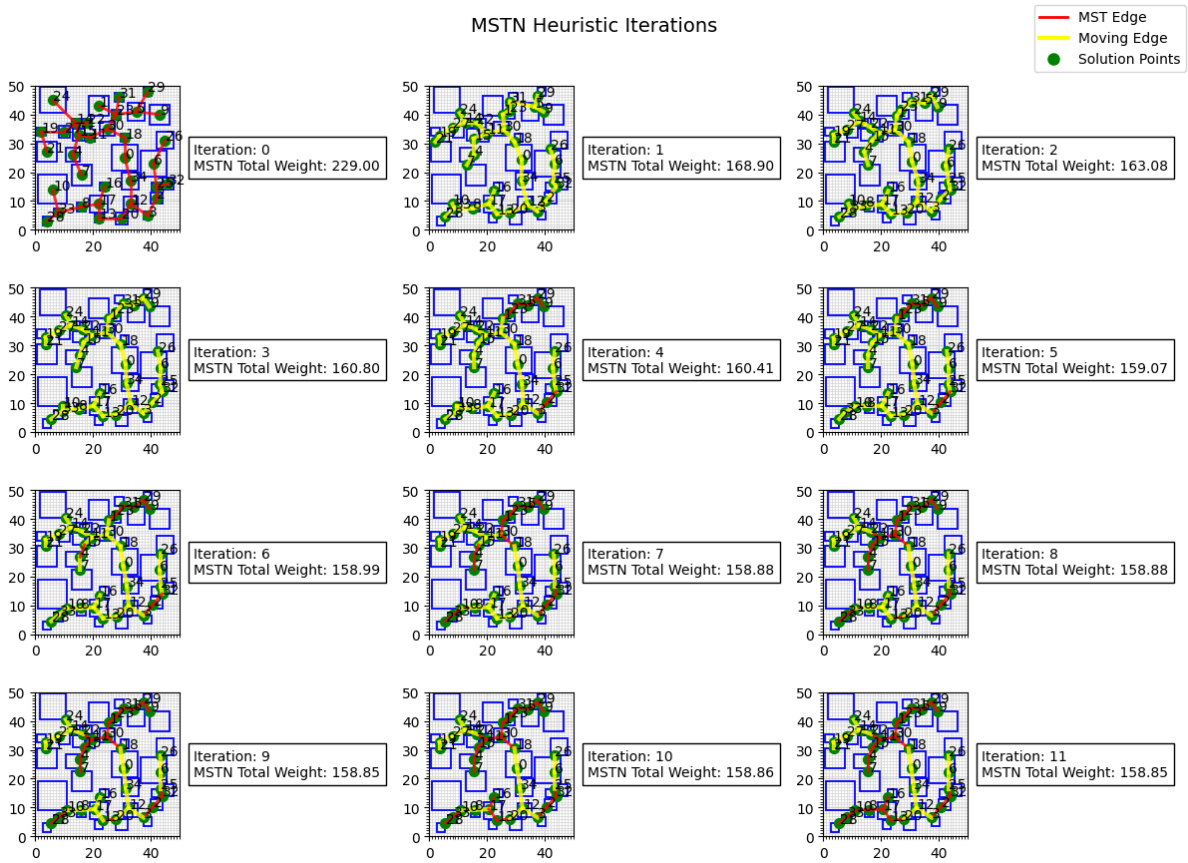

Figure 3: Heuristic iterations for one instance. Each subplot shows one step: representatives (green), edges (red/yellow), and total cost per iteration.

This configuration was used to validate and debug the algorithm in the integer, non-overlapping setting before shifting to more general cases.

A summary of the result:

- The initial MSTN cost (at centers) is 229.00.
- Rapid cost drop to 168.90 on first iteration.
- Further improvement to final cost 158.86 over 12 iterations (≈30.6% reduction).
- The final solution (Figure 4) shows representatives optimally repositioned on square boundaries, minimizing edge costs.

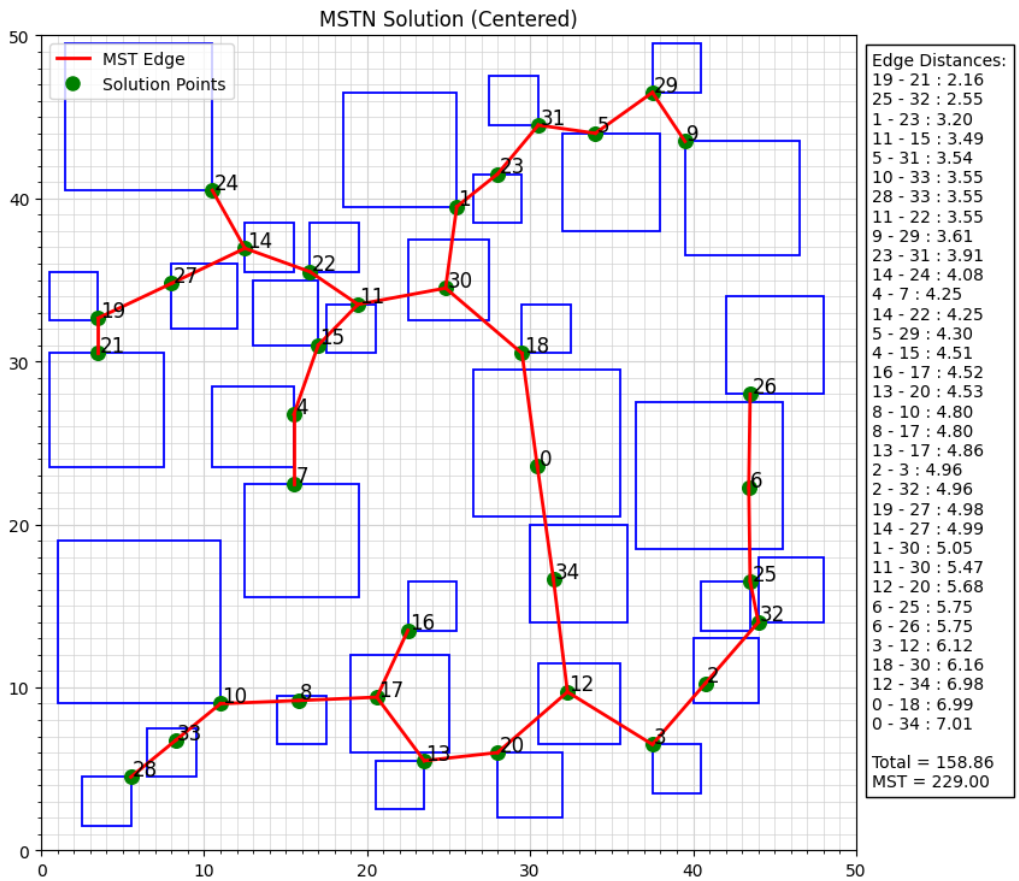

Figure 4: Final MSTN solution for the instance: green dots are optimized representatives, red edges the final MST. Cost = 158.86.

5.2 Scaling and effectiveness: statistical evaluation over many instances

We also assessed the heuristic across hundreds of random instances, with the following protocol:

- For each experiment, a grid size was chosen from 1 to 865.
- The number of neighborhoods n was set as $n = \lfloor 0.8 \times \text{grid size} \rfloor$.
- For each neighborhood, squares were assigned a random side length uniformly sampled in $[1\%, 3\%]$ of the grid size, i.e., side $\in [0.01 \times \text{grid size},\ 0.03 \times \text{grid size}]$.

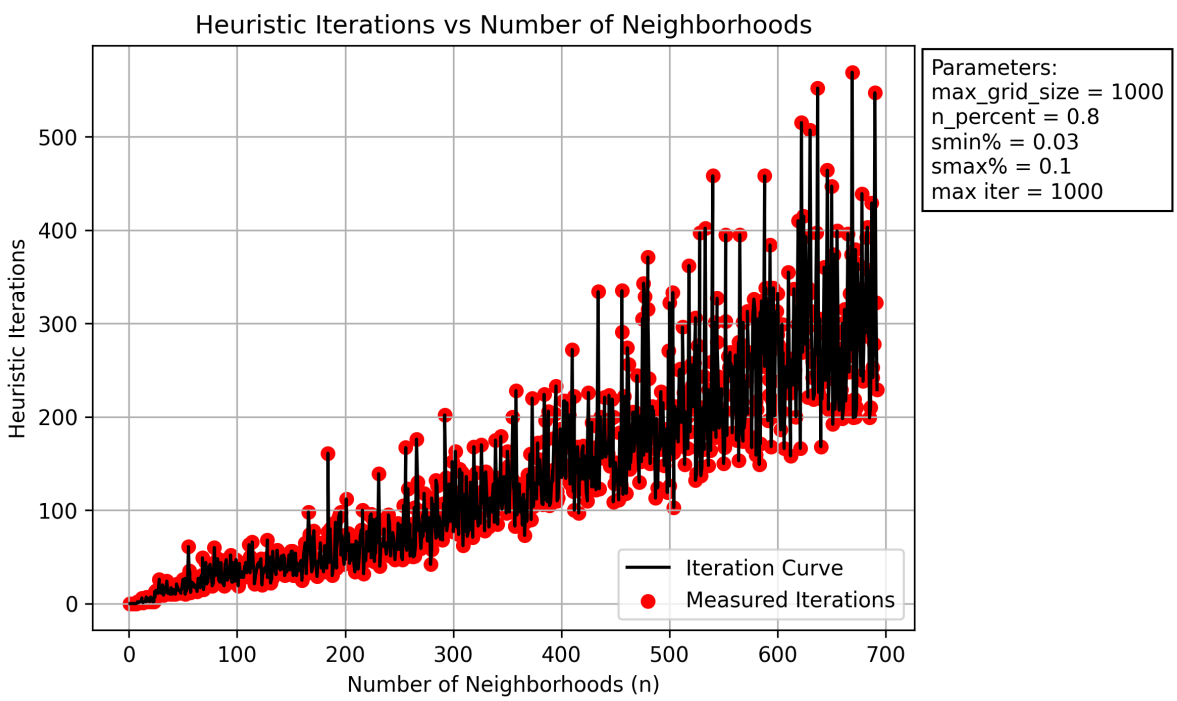

Figure 5: Heuristic iterations to convergence vs. number of neighborhoods n. Each point is a separate instance. Parameters in pane.

Global trends:

- **Iterations:** The number needed for convergence remains moderate as n increases, reflecting the efficiency of the alternating updates (see Figure 5).
- **Runtime:** The method is practical for hundreds of neighborhoods, thanks to the fast contraction per iteration (see Figure 6).
- **Reduction:** Substantial and consistent MSTN cost gains (often 30–40%) across all tested regimes (see Figure 7).

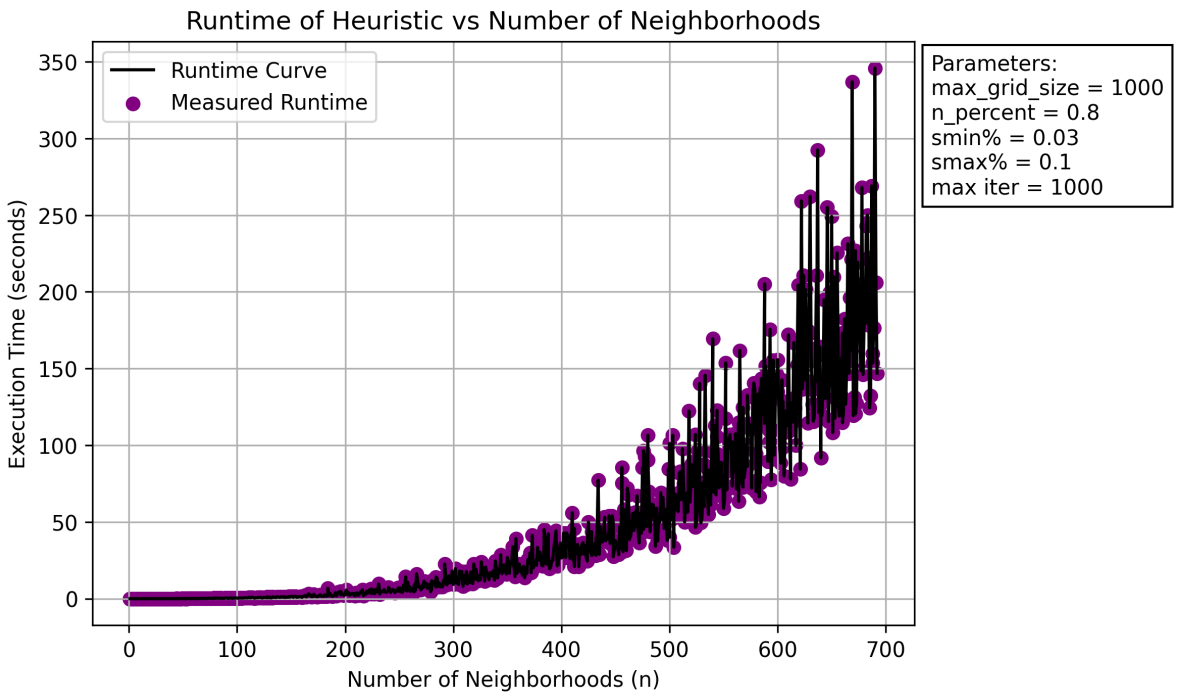

Figure 6: Total runtime (seconds) vs. number of neighborhoods, over all tested grid sizes and corresponding n.

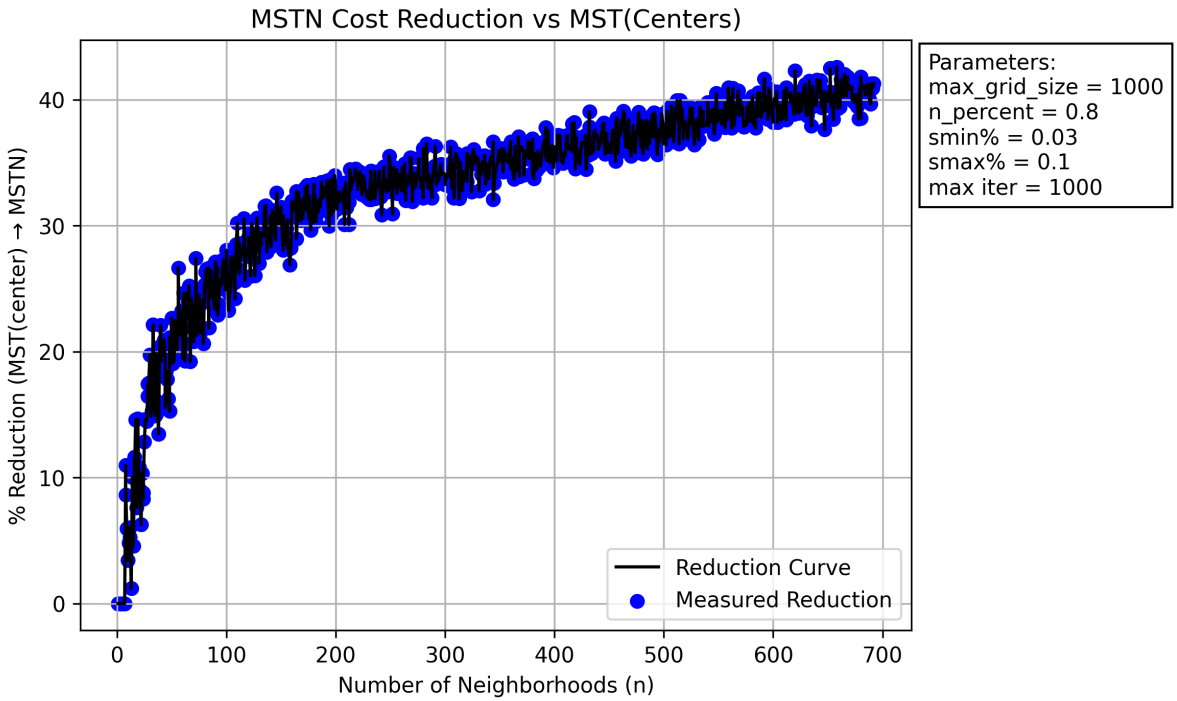

Figure 7: Percentage reduction in tree cost (MST on centers vs. MSTN) as n increases.

Heuristic Design: The update mechanism is inspired by classification and clustering algorithms, especially k-means: each step performs a barycentric update followed by projection, quickly recentering representatives in analogy with centroid assignment in cluster analysis.

Time complexity: The overall time complexity of the heuristic is dominated by the repeated computation of the minimum spanning tree at each iteration. Each call to Kruskal's algorithm runs in $\mathcal{O}(|E| \log |E|)$ time (where E is the set of edges) [1]. Additionally, updating the node positions requires $\mathcal{O}(|V|)$ operations per iteration (where V is the set of nodes). Therefore, the per-iteration complexity is

$$\mathcal{O}(|E| \log |E| + |V|).$$

If the number of iterations is K, the total complexity of the heuristic is

$$\text{Total complexity} = K \cdot \mathcal{O}(|E| \log |E| + |V|).$$

In practice, for moderately dense graphs, $|E| \log |E| \gg |V|$, so the complexity can be simplified to

$$\text{Total complexity} = \mathcal{O}(K\, |E| \log |E|).$$

This shows that the heuristic remains highly efficient and scalable for practical instance sizes.

Conclusion: The heuristic yields rapid, substantial improvements over classical MST cost, and scales robustly as the grid size and number of neighborhoods grow, validating its practical applicability for large geometric networks.

6 MINLP formulations overview

This section briefly summarizes the two main MINLP formulations for MSTN described in [1] and their computational behavior. These formulations provide exact methods for solving MSTN to optimality, and serve as a benchmark for evaluating the quality of heuristic solutions.

6.1 Mathematical formulation

Following [1], MSTN can be formulated as a mixed-integer nonlinear program where each vertex $v \in V$ is associated with a continuous decision variable $y_v \in N_v \subset \mathbb{R}^d$ representing the chosen representative inside its convex neighborhood. For each edge $e \in E$, a binary variable $x_e \in \{0, 1\}$ indicates whether e belongs to the spanning tree, and a continuous auxiliary variable $u_e \geq 0$ satisfies

$$u_{\{i,j\}} \geq \|y_i - y_j\|, \qquad \text{for } e = \{i, j\} \in E.$$

In order to linearize the objective function, the RL–MSTN formulation introduces a variable $\theta_e = u_e x_e$, and a valid upper bound $\tilde{U}_e$ on u_e. Thus, the objective becomes linear:

$$\min \sum_{e \in E} \theta_e.$$

The relationship $\theta_e = u_e x_e$ is enforced through McCormick inequalities, labeled as in [1]:

$$\theta_e \le \tilde{U}_e x_e, \tag{U1}$$

$$\theta_e \ge u_e - \tilde{U}_e(1 - x_e), \tag{U2}$$

combined with

$$\theta_e \le u_e, \qquad \theta_e \ge 0, \qquad u_e \ge 0.$$

Let $ST(G)$ denote the spanning tree polytope of G. The complete RL–MSTN model is thus:

$$\begin{aligned}
\min \quad & \sum_{e \in E} \theta_e \\
\text{s.t.} \quad & x \in ST(G), \\
& y_v \in N_v, \qquad v \in V, \\
& u_{\{i,j\}} \ge \|y_i - y_j\|, \qquad \{i,j\} \in E, \\
& \theta_e \le \tilde{U}_e x_e, && \forall e \in E, \\
& \theta_e \ge u_e - \tilde{U}_e(1 - x_e), && \forall e \in E, \\
& \theta_e \le u_e, \quad \theta_e \ge 0, \quad u_e \ge 0, && \forall e \in E, \\
& x_e \in \{0, 1\}, && \forall e \in E.
\end{aligned}$$

If the neighborhoods N_v are second-order-cone representable (e.g., Euclidean balls or axis-aligned boxes) and distances are modeled using a rational q-norm, both the norm constraints and the membership constraints $y_v \in N_v$ admit SOC or linear descriptions. Consequently, RL–MSTN has a linear objective, binary combinatorial constraints, and convex conic inequalities, and can be tackled by modern conic branch-and-cut solvers.

6.2 SEC-based formulation

The first formulation uses the classical representation of the spanning tree polytope via subtour elimination constraints. The binary variables x_e for $e \in E$ must satisfy

$$\sum_{e \in E} x_e = |V| - 1,$$

$$\sum_{\{i,j\} \in E : i,j \in S} x_{\{i,j\}} \le |S| - 1, \quad \emptyset \ne S \subset V.$$

The first equation enforces that exactly $n-1$ edges are chosen; the second family of inequalities eliminates cycles by forbidding more than $|S| - 1$ edges within any proper subset S of vertices.

Combined with the norm constraints linking u_e and y, and with McCormick linearization of the bilinear products $u_e x_e$, this yields a (large) MINLP that can be solved by off-the-shelf solvers, typically using branch-and-cut to separate violated subtour constraints dynamically. Computational experiments in [1] show that this SEC-based formulation is relatively strong and tends to outperform MTZ on many instances, at the price of managing an exponential family of cuts.

6.3 MTZ-based formulation

The second formulation replaces the SEC description by an MTZ-style model. An arbitrary root vertex is selected, and each undirected edge is given two possible orientations with binary arc variables. Additional continuous "order" variables are introduced to prevent subtours by enforcing a topological order along arcs. The resulting MILP formulation is compact, with a polynomial number of constraints, but its linear relaxation is generally weaker.

In the MSTN context, the MTZ constraints are combined with the same continuous variables y_v and u_e, and the same linearization of the objective. Blanco et al. report that while MTZ can solve many smaller instances, it is dominated by the SEC-based formulation for more difficult cases, especially when neighborhood radii grow and intersections become frequent [1].

6.4 Decomposition and mathheuristic

Because MSTN is biconvex, [1] also propose a Benders-like decomposition and a mathheuristic that alternate between solving convex subproblems in (y, u) for a fixed tree and solving MST subproblems in x for fixed edge weights. The decomposition is used as an exact method, whereas the mathheuristic drops optimality guarantees in favor of speed and scalability: it can handle much larger instances than the raw MINLP formulations, at the cost of obtaining only near-optimal solutions.

The mathheuristic proposed in [1] is conceptually similar to the alternating heuristic implemented in this project, but uses exact convex optimization solvers at each iteration rather than the simple projected averaging rule. Their results show that the mathheuristic can solve instances with up to 100 vertices and achieve solutions within a few percent of optimality in reasonable time.

6.5 Relationship to the implemented heuristic

The heuristic implemented in this project can be seen as a simplified version of the mathheuristic proposed in [1]. In the latter, for a fixed tree x, the continuous subproblem in (y, u) is solved exactly as a convex optimization problem (specifically, a second-order cone program), and for fixed continuous variables, MSTs are recomputed exactly. The heuristic in this project replaces the exact convex solve by a single projected averaging step, which is much cheaper but does not guarantee optimality in the continuous subspace.

Despite this simplification, the general philosophy is the same: exploit the biconvex structure of MSTN by alternating between solving an MST in edge space and improving representative positions in geometric space. This connection motivates possible future improvements to the current implementation, such as:

- replacing the barycenter projection by the exact solution of a small convex subproblem for each vertex or for the whole tree;
- incorporating random restarts or perturbations to escape poor local minima;
- using bounds or cuts inspired by the Benders decomposition of [1].

7 Future work: learning and advanced heuristics

The MSTN problem offers many opportunities for further research and computational improvement. Several directions are particularly appealing in the context of this project.

7.1 More advanced mathheuristics

The existing heuristic can be enhanced by ideas from [1] and from the broader mathheuristic literature:

- **Multi-start strategies:** generate several different initial trees (for example, MSTs under different distance metrics or random perturbations) and run the alternating heuristic from each, keeping the best solution.
- **Local search in tree space:** after convergence of the positions, attempt local modifications of the tree (such as edge swaps) that decrease the total length, updating positions accordingly.
- **Adaptive step sizes:** instead of moving representatives directly to the projected barycenter, use partial steps (e.g., $y_i^{(k+1)} = y_i^{(k)} + \alpha(z_i - y_i^{(k)})$ where z_i is the projected barycenter and $\alpha \in (0, 1]$) to improve stability and possibly convergence speed.
- **Variable neighborhood descent:** combine the alternating heuristic with local search moves that modify the tree structure, such as adding and removing edges, to explore a broader solution space.

7.2 Machine learning for heuristic design

An intriguing direction is to use machine learning techniques to automatically learn good heuristic policies for MSTN from data. Possible approaches include:

- **Learning update rules.** Instead of a fixed rule "move each point to the projected average of its neighbors", train a model that, given the current local configuration (neighborhood shape, neighbor positions, edge lengths), predicts a promising direction or target point inside the neighborhood. This could be implemented using supervised learning on data generated by solving small instances to optimality, or using reinforcement learning to directly optimize the heuristic policy.
- **Learning tree construction.** Use reinforcement learning or graph neural networks to learn policies that construct good trees given the pairwise distances between neighborhoods, possibly imitating high-quality solutions from an exact solver on small instances. Recent work on neural combinatorial optimization for the traveling salesman problem and vehicle routing suggests that such approaches can be effective [1].
- **Parameter tuning.** Use data-driven methods (e.g., Bayesian optimization, evolutionary algorithms) to tune hyperparameters of the heuristic (step sizes, tolerances, restart strategies, projection methods) for different classes of instances. This could

be done offline by generating a large dataset of instances and running many heuristic variants, or online by adapting parameters during the search.

- **Learning to predict solution quality.** Train a model to predict, given an instance's features (number of vertices, neighborhood sizes, overlap statistics), the expected performance of different heuristics or initialization strategies. This meta-learning approach could guide algorithm selection and configuration.

These ideas build on recent successes of learning-based methods for combinatorial optimization, and could be adapted to the MSTN setting with relatively modest effort. The biconvex structure of MSTN may even make it easier to apply learning, since the continuous subproblems are well-behaved and could serve as targets for supervised learning.

8 Conclusion

This report has addressed the Minimum Spanning Tree with Neighborhoods (MSTN) problem, reviewing its mathematical formulations, algorithmic challenges, and diverse solution strategies. In addition to summarizing MINLP models, we have proposed and implemented a practical heuristic, inspired by clustering techniques such as k-means, that exploits the biconvex structure of the MSTN model.

By alternating between minimum spanning tree computations and projected barycentric updates for representatives, the heuristic converges rapidly to feasible solutions that substantially reduce total tree length. A detailed walkthrough on a 35-neighborhood instance demonstrated over 30% cost reduction compared to the naive MST on region centers.

Beyond individual examples, extensive experiments on random instances of varying size and density—summarized with new global plots—demonstrate several key properties:

- The method scales robustly: both iterations and runtime remain moderate as the problem grows (up to hundreds of neighborhoods).
- Substantial cost reductions over center-based MST are observed consistently, often exceeding 30–40%.
- The heuristic's time complexity is linear in the number of iterations, each dominated by a Kruskal run ($\mathcal{O}(|E| \log |E|)$ per iteration).
- The k-means-inspired update mechanism leads to prompt contraction and fast convergence not only on small, simple instances but also on large and challenging problems with overlapping or real-valued neighborhoods.
- Incremental development—beginning with integer, non-overlapping squares—enabled rapid extension to more general geometric cases.
- In contrast, exact MINLP approaches only handle small/medium graphs due to combinatorial explosion.

This approach closes the gap between theory and scalable heuristics, offering a practically efficient tool for designing geometric network structures under uncertainty. The results also set the groundwork for future research: further sophistication of the mathheuristic, tighter integration of machine learning for adaptive update rules or instance-based algorithm selection, and extensions to broader classes of combinatorial/continuous network design problems.

Bibliography

References

[1] Víctor Blanco, Elena Fernández, Justo Puerto. *Minimum Spanning Trees with neighborhoods: Mathematical programming formulations and solution methods.* European Journal of Operational Research, 262 (2017), 863–878.

Acknowledgments

Portions of this work were assisted using the AI system ChatGPT developed by OpenAI, which was used for code suggestions, debugging assistance, and explanatory support. The final analysis, results, and conclusions presented here are solely my own.